



Testing

Programm für heute



QA|WARE

Vorlesung

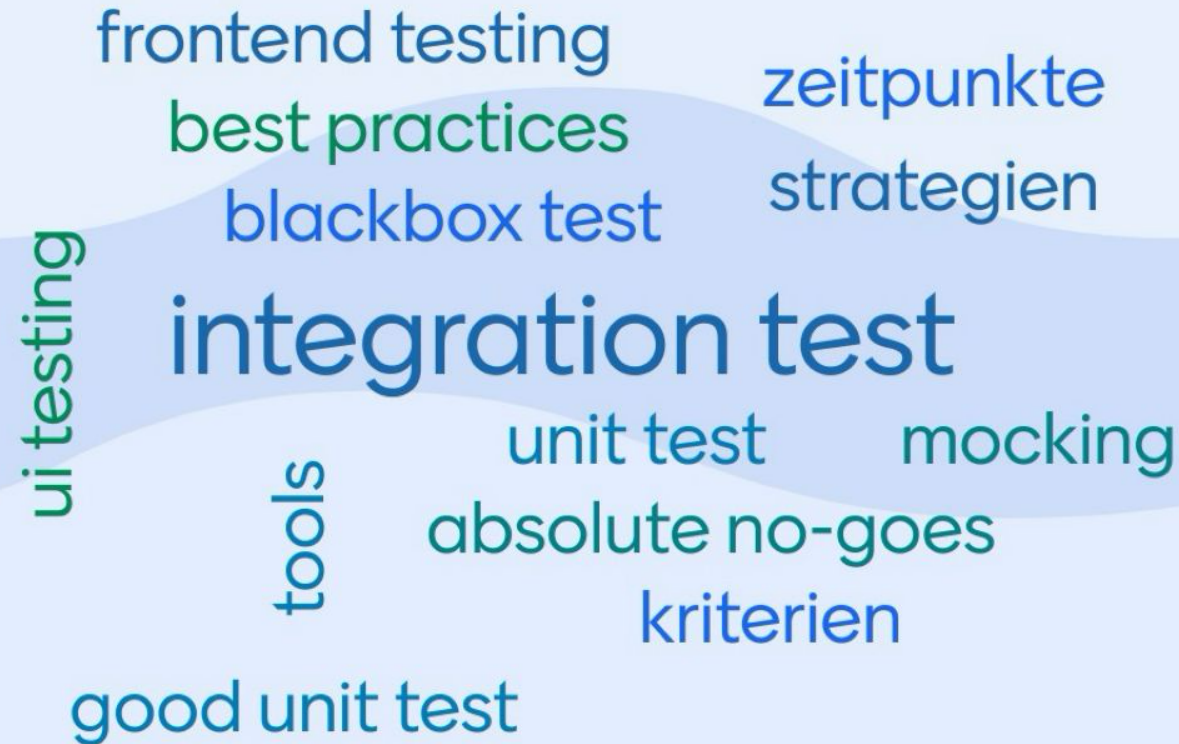
1. Einführung & Motivation
2. Best Practices
3. Github Actions

Übung

1. Library Service um Tests erweitern mit Mockito
2. Github Pipeline bauen

Was würdet ihr gerne zum Thema Testen wissen?

14 responses



A word cloud visualization of responses to the question 'Was würdet ihr gerne zum Thema Testen wissen?'. The words are arranged in a light blue, wavy background. The most prominent word is 'integration test' in the center. Other words include 'frontend testing', 'best practices', 'blackbox test', 'zeitpunkte', 'strategien', 'ui testing', 'unit test', 'mocking', 'absolute no-goes', 'kriterien', 'tools', and 'good unit test'.

frontend testing
best practices
blackbox test
zeitpunkte
strategien
integration test
ui testing
unit test
mocking
absolute no-goes
kriterien
tools
good unit test

RECAP



QA|WARE

Qualitätskosten

Je früher wir Fehler erkennen, desto billiger ist es diese zu beheben.

RECAP



QA|WARE

Welchen Unterschied macht es in Bezug auf die Kosten einen Fehler bereits bei der Anforderungsanalyse im Vergleich zur Live-Anwendung zu finden?

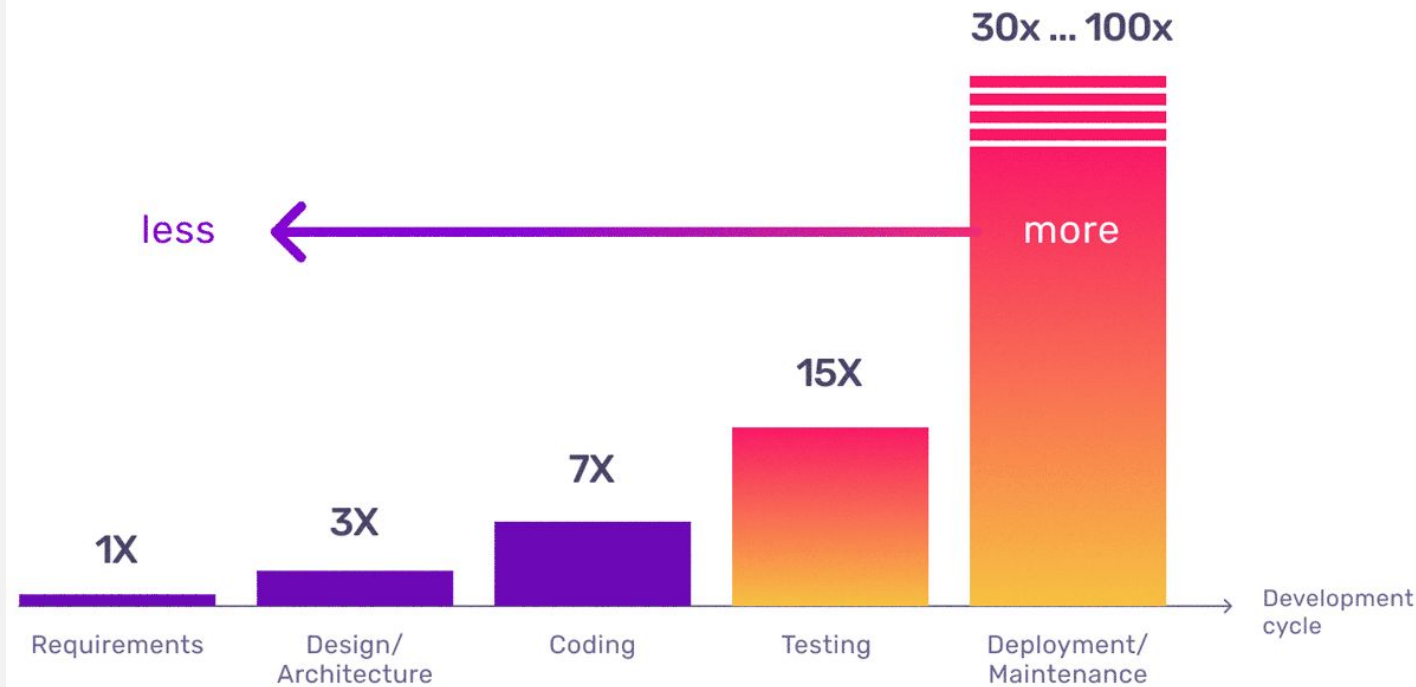
Je früher wir Fehler erkennen, desto billiger ist es diese zu beheben.

RECAP



QA|WARE

Cost of Defects



Die Gründe für die steigenden Kosten sind vielfältig.

RECAP



QA|WARE

- Test- und Deploymentsetup ist bereits geschrieben und muss bei Änderungen im Zweifel komplett angepasst werden
- Je tiefliegender die Fehler sind, desto mehr andere Stellen sind von Änderungen betroffen
- Große Refactorings und Bugfixes produzieren eventuell neue Probleme
- Kleine Fehler an grundlegenden Komponenten oder anfänglich falsch getroffene Designentscheidungen führen oft zu massiven Problemen
- Entwickler:innen sind nicht mehr so tief im Thema und müssen sich “neu reindenken”

Qualitativ hochwertig bedeutet nicht: fehlerfrei.

RECAP



QA|WARE

- Software ohne Fehler ist unrealistisch aufgrund:
 - Kosten/Nutzenverhältnis bei der Erstellung des Systems
 - Fehlerfreiheit setzt vollständig spezifiziertes Verhalten voraus
 - Vollständiger Test aller Programmpfade unmöglich
 - Korrektheitsbeweise sind nur für einfache System möglich

Ziel muss sein:

Kritische Ausfälle vermeiden. Die Kritikalität hängt dabei vom jeweiligen System ab.

Fehlerfreie Software ist unrealistisch und nicht kosteneffizient.



QA|WARE

Testing shows the presence, not the absence of bugs.

Dijkstra, 1969

Fail fast. Fail often.

RECAP

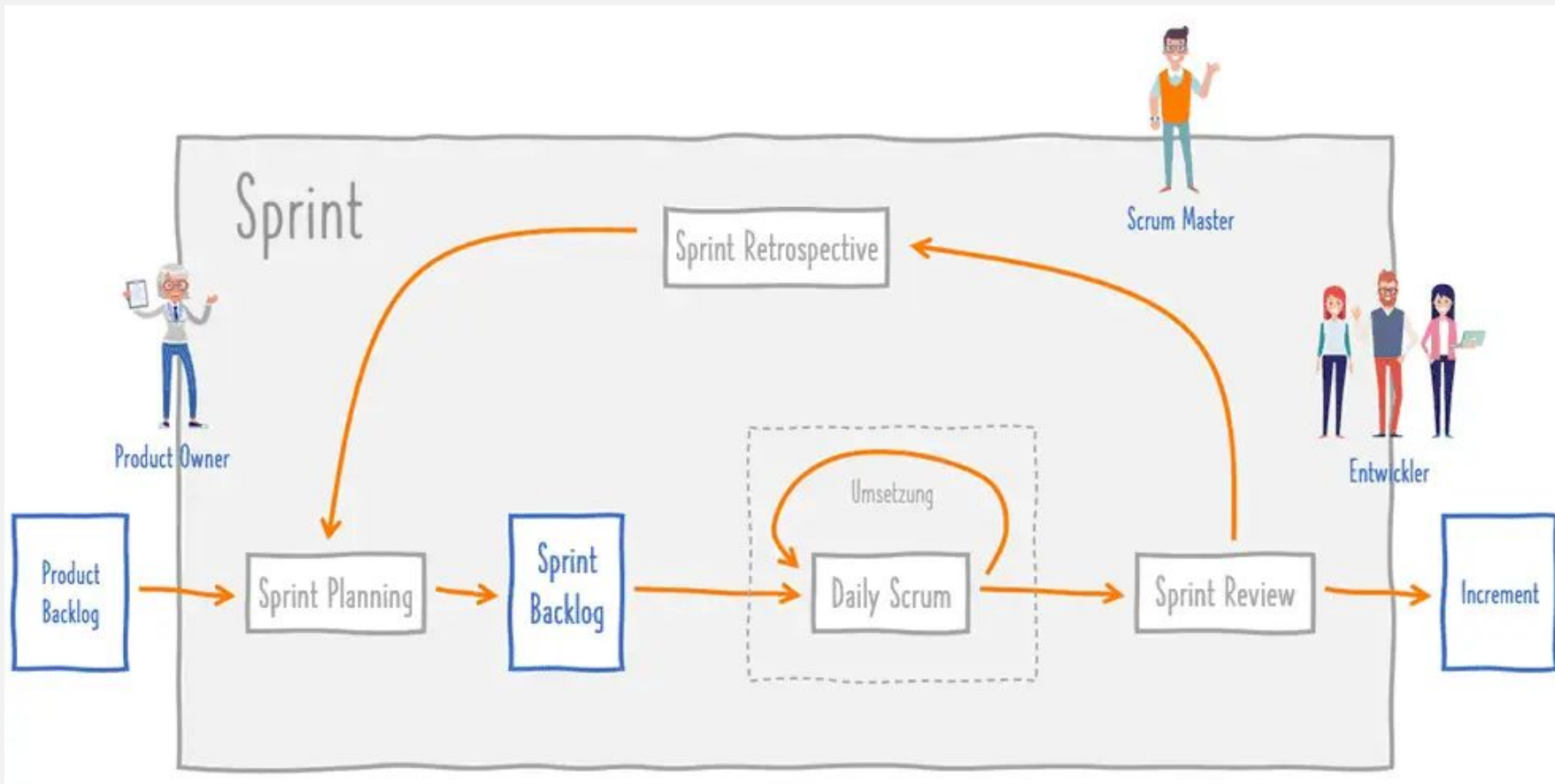


QA|WARE

Agiles Arbeiten hilft dabei Fehler kontinuierlich zu erkennen und zu beheben.



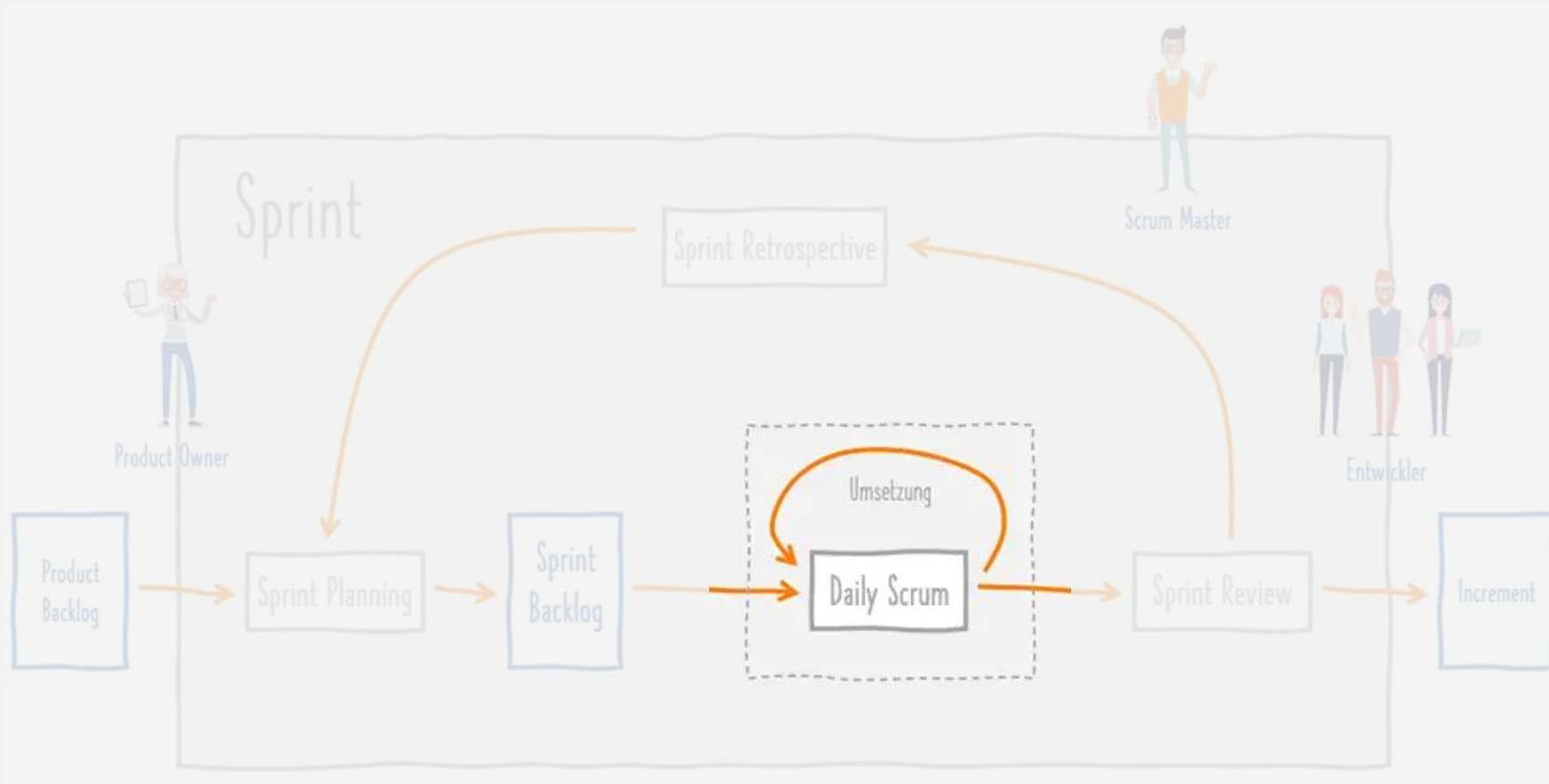
QA|WARE



Wir sehen uns heute einen Teil des Daily Scrum an.



QA|WARE



Daily Scrum besteht aus einer Reihe von Tätigkeiten.



QA|WARE

- Neue Anforderungen an das Projekt mit den Kunden:innen absprechen
- Featurespezifikationen schreiben
- Feature umsetzen und Tests schreiben
- Featurereview durchführen
- Feature anliefern
- ...



QA|WARE

Motivation

Wieso schreiben wir überhaupt Tests?



QA|WARE

- Damit es im Build kracht, und nicht nachts in Produktion
 - **DU** wirst angerufen
- Refactoring gefahrlos möglich
- Benutzer der eigenen API
- Dokumentation der Verwendung der API
- Bugs werden früh gefunden, damit billige Behebung
- Peinliche Fehler werden von dir gefunden, nicht im Code Review
- Schnelle Verifikation, ob der Code tut was er tun soll

Fail fast. Fail often.

RECAP



QA|WARE



QA|WARE

Testarten

Wir kennen viele Arten von Tests. Die Benennung ist nicht immer eindeutig und die Trennung nicht immer scharf.



QA|WARE

- Unit Test
- Integrationstest (= Komponententests)
- Systemtest (= Integrationstest = E2E Test = Functional Test)
- UI-Test
- Lasttest (=Performancetest)
- Penetration Test
- Abnahmetest
- Infrastruktur-Tests
- Contract Based Test (Schnittstellentest)
- Automatische Überprüfung von Lizenzen, KPIs, ...
- Regressionstest
- Manuelle Tests
- ...

Wir kennen viele Arten von Tests. Die Benennung ist nicht immer eindeutig und die Trennung nicht immer scharf.



QA|WARE

■ Unit Test

- Integrationstest (= Komponententests)
- Systemtest (= Integrationstest = E2E Test = Functional Test)
- UI-Test
- Lasttest (=Performancetest)
- Penetration Test
- Abnahmetest
- Infrastruktur-Tests
- Contract Based Test (Schnittstellentest)
- Automatische Überprüfung von Lizenzen, KPIs, ...
- Regressionstest
- Manuelle Tests

Unit Test

- testet einzelne Klassen, Funktionen, Algorithmen, ... isoliert
- schnell ausführbar
- hat keine Abhängigkeiten
- läuft immer im Build mit
- heißt manchmal auch Modul- oder Komponententest

Wir kennen viele Arten von Tests. Die Benennung ist nicht immer eindeutig und die Trennung nicht immer scharf.



QA|WARE

- Unit Test
- **Integrationstest (= Komponententests)**
- Systemtest (= Integrationstest = E2E Test = Functional Test)
- UI-Test
- Lasttest (=Performancetest)
- Penetration Test
- Abnahmetest
- Infrastruktur-Tests
- Contract Based Test (Schnittstellentest)
- Automatische Überprüfung von Lizenzen, KPIs, ...
- Regressionstest
- Manuelle Tests

Integrationstest = Komponententest

- testet mehrere Klassen oder Komponenten und deren Interaktion
- testet die Interaktion mit DBs, Schnittstellen, Frameworks, ...
- Ziel ist oft, Fehler in der Kommunikation zwischen Komponenten zu finden

Wir kennen viele Arten von Tests. Die Benennung ist nicht immer eindeutig und die Trennung nicht immer scharf.



QA|WARE

- Unit Test
- Integrationstest (= Komponententests)
- **Systemtest (= Integrationstest = E2E Test = Functional Test)**
- UI-Test
- Lasttest (=Performancetest)
- Penetration Test
- Abnahmetest
- Infrastruktur-Tests
- Contract Based Test (Schnittstellentest)
- Automatische Überprüfung von Lizenzen, KPIs, ...
- Regressionstest
- Manuelle Tests

Systemtest = Integrationstest = E2E
Test = Functional Test

- testet das gesamte System
- hierfür wird oft die ganze Anwendung im Test hochgefahren
- alternativ wird gegen eine laufende Integrationsumgebung getestet
- Externe Systeme werden entweder gemockt oder aufgerufen

Wir kennen viele Arten von Tests. Die Benennung ist nicht immer eindeutig und die Trennung nicht immer scharf.



QA|WARE

- Unit Test
- Integrationstest (= Komponententests)
- Systemtest (= Integrationstest = E2E Test = Functional Test)
- **UI-Test (= E2E Test)**
- Lasttest (=Performancetest)
- Penetration Test
- Abnahmetest
- Infrastruktur-Tests
- Contract Based Test (Schnittstellentest)
- Automatische Überprüfung von Lizenzen, KPIs, ...
- Regressionstest
- Manuelle Tests

UI-Tests

- testen die UI der Anwendung
 - entweder isoliert für einzelne Komponenten (-> Unit Test, Integrationstest)
 - oder simulieren Nutzer-Interaktionen in der laufenden Anwendung und testen damit das ganze System (-> Systemtest)
- werden oft vernachlässigt oder vergessen :(
- z.B. mit Browserautomatisierung (Selenium), Screenshot Testing
- oft bietet sich hier Behavior Driven Testing an

Wir kennen viele Arten von Tests. Die Benennung ist nicht immer eindeutig und die Trennung nicht immer scharf.



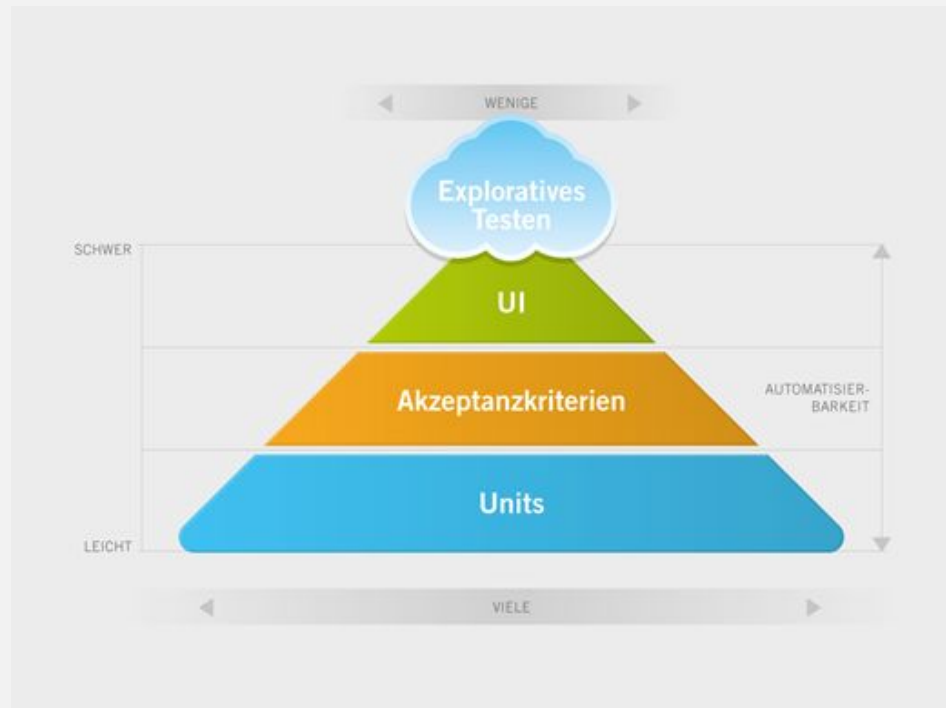
QA|WARE

- Unit Test
- Integrationstest (= Komponententests)
- Systemtest (= Integrationstest = E2E Test = Functional Test)
- UI-Test
- Lasttest (=Performancetest)
- Penetration Test
- Abnahmetest
- Infrastruktur-Tests
- Contract Based Test (Schnittstellentest)
- Automatische Überprüfung von Lizenzen, KPIs, ...
- **Regressionstest**
- Manuelle Tests

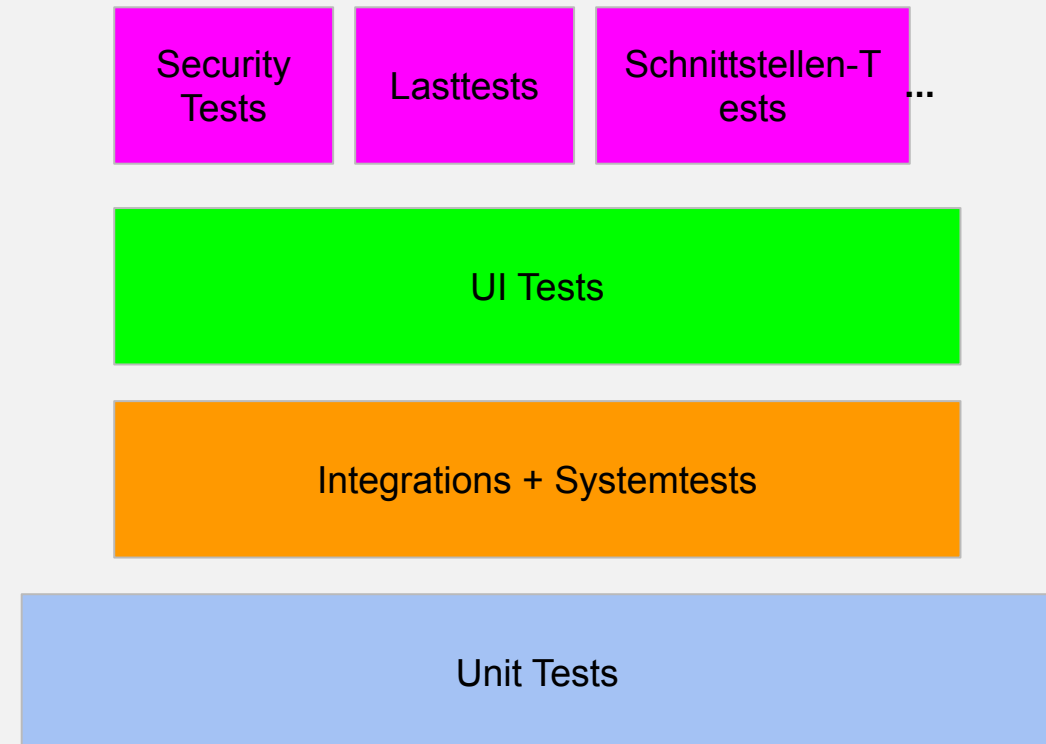
Regressionstest

- ist eigentlich keine eigene "Testart"
- stellt sicher, dass Bestandsfunktionalität nicht kaputt geht
- ein Test für ein neues Feature wird nach Abschluss automatisch zum Regressionstest

Alle Ebenen dieser Tests sind mittlerweile gut automatisierbar - wir müssen sie nur schreiben.



Testpyramide



Was ist also dieses Test Driven Development?



QA|WARE

- Ganz streng genommen schreibst du zuerst den Unit Test.
- Erst dann implementierst du dein Feature, bis der Test grün ist. Und zwar nur genau so viel!
- Danach refactorst du deinen Code.
- Dann beginnt das Ganze wieder von vorne.

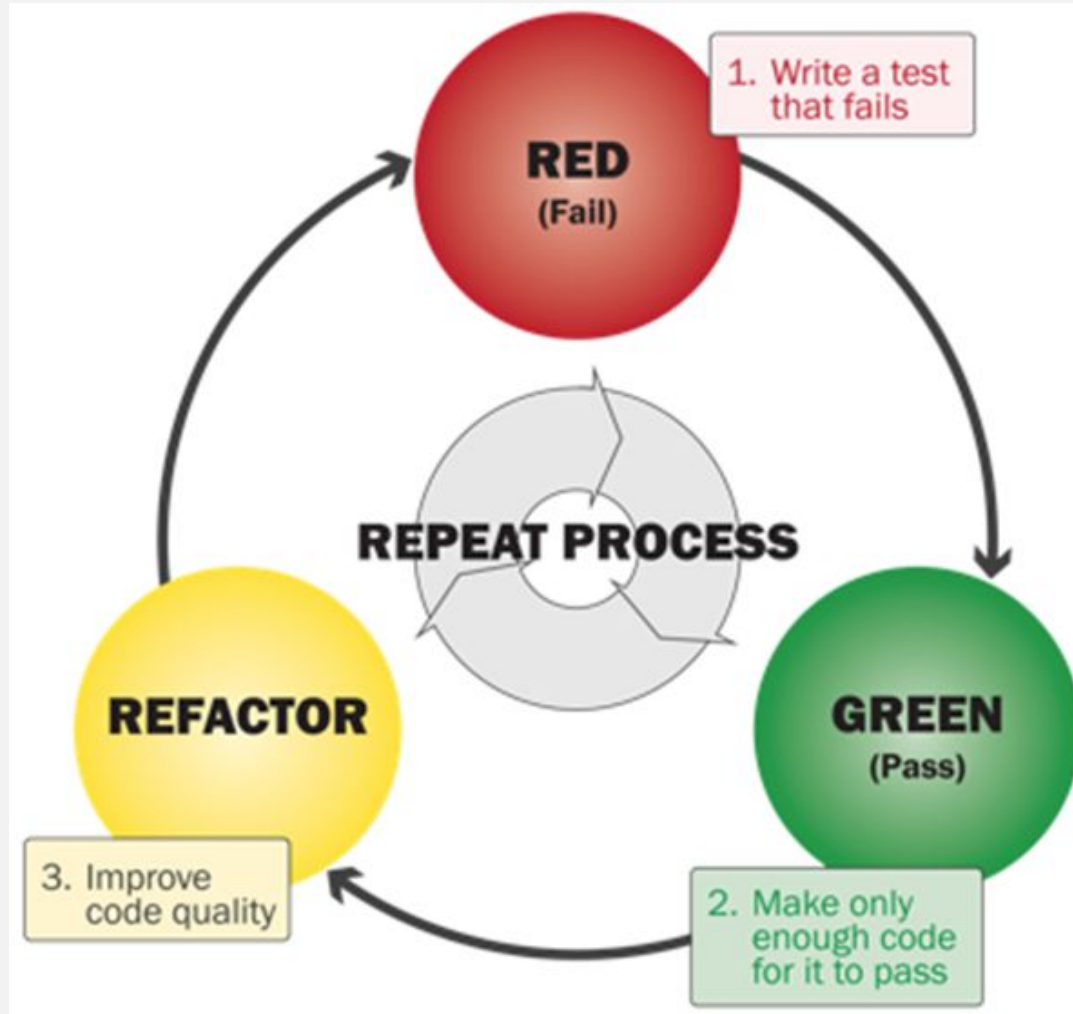
- Damit hat der Unit Test einen großen Einfluss auf die tatsächliche Implementierung.
- Es soll Leute geben, denen dieses Vorgehen nicht so super viel Spaß macht...

- Folgendes Vorgehen hat sich unserer Meinung nach bewährt:
Schreibt mindestens eure Unit Tests parallel zum Feature, also: Code schreiben -> Test schreiben -> Code schreiben -> Test schreiben ...

Was ist also dieses Test Driven Development?



QA|WARE



Fail fast. Fail often.

RECAP



QA|WARE



QA|WARE

Testbausteine

Um eine Anwendung zu testen bedarf es einiger Bausteine.



QAWARE

- Natürlich brauchen wir Code, den wir testen wollen.
- Wir brauchen den Testcode, der unseren Produktivcode testet.
- Für größere Anwendungen brauchen wir einen Testdatengenerator (oder statische Testdaten – wie immer: it depends!)
- Wir brauchen eine Testautomatisierung.
- Auch unser Testkonzept muss dokumentiert sein!



QA|WARE

Best practices



QA|WARE

Single Responsibility Principle

Nicht jeder Code ist problemlos testbar.



QA|WARE

- Komponenten müssen so designed sein, dass sie testbar sind
- Probleme:
 - Globaler Zustand (Singletons, static, System.out ...)
 - “new” als Abhängigkeiten in der Komponente
 - Zu viele Zuständigkeiten einer Klasse

Eine Lösung dafür ist Dependency Injection.



QA|WARE

- Abhängigkeiten der Klasse identifizieren (Halte nach “new” und Singletons Ausschau)
- Aus Abhängigkeiten Interfaces extrahieren
- Abhängigkeiten in die Klasse per Konstruktor übergeben
- Diese Interfaces kann man in Tests mocken / implementieren

Beispiel: vorher.

```
public class Car {  
    private final TireImpl frontRight = new TireImpl();  
    private final TireImpl frontLeft = new TireImpl();  
    private final TireImpl backRight = new TireImpl();  
    private final TireImpl backLeft = new TireImpl();  
  
    public void drive() {  
        frontRight.roll();  
        frontLeft.roll();  
        backRight.roll();  
        backLeft.roll();  
    }  
}
```



QAWARE

Beispiel: nachher.

```
public class CarDI {  
    private final Tire frontRight;  
    private final Tire frontLeft;  
    private final Tire backRight;  
    private final Tire backLeft;  
  
    public CarDI(Tire frontRight, Tire frontLeft, Tire backRight, Tire backLeft) {  
        this.frontRight = frontRight;  
        this.frontLeft = frontLeft;  
        this.backRight = backRight;  
        this.backLeft = backLeft;  
    }  
  
    public void drive() { ...  
}
```



QAWARE



QA|WARE

Unit Tests

Was macht gute (Unit) Tests aus?



- Sie sind **korrekt** implementiert und **testen** tatsächlich die Funktionalität.
- Tests müssen **schnell** laufen. Sonst schaltest du sie sowieso aus. Ausnahmen sind erlaubt für Tests, die nicht in jedem Build ausgeführt werden müssen.
- Sie müssen **einfach** verständlich und lesbar sein.
- Außerdem muss ein Test **unabhängig** sein
 - von anderen Tests
 - und sonstigen Dingen (Betriebssystem, Tageszeit, ...)
- **Offline**-Lauffähigkeit sollte sichergestellt sein.
- Der Testcode muss **wartbar** sein, es gelten die gleichen Styling, Naming, ... Vorgaben wie für den Produktivcode.
- Jeder Test muss einfach **ausführbar** sein für jeden Entwickler. Komplizierte Test-Setups nerven jeden von uns.

Wie kriegen wir das hin?



QA|WARE

Im Code selbst:

- Behandle den Testcode so wie den Produktivcode
 - Codestyle
 - zu lange Methoden refactoren
 - sinnvolle Namen
 - ...
- Stelle sicher, dass dein Test auch wirklich etwas testet:
 - z.B. über Assertions
 - ein Test ohne Assertion ist immer verdächtig
- Verwende keine Logausgabe oder `System.out.println(...)` im Test und vergleiche dann die Log Ausgaben. Verwende lieber Assertions.

Wie kriegen wir das hin?



QA|WARE

In der Teststruktur:

- Überlege dir eine sinnvolle Struktur für deine Tests. Folgendes hat sich z.B. bewährt
 - 1. Setup: initialisierte deinen Objekte
 - 2. Code ausführen
 - 3. Ergebnis mit dem erwarteten Ergebnis vergleichen
 - Hierbei hilft dir z.B. Spock durch die vorgegebene Struktur
- Dokumentiere dein Testkonzept

Ein Beispiel mit Groovy.



QAWARE

```
import spock.lang.Specification

class CalculatorSpec extends Specification {

    def "addition sollte korrekt sein"() {
        given: "ein einfacher Taschenrechner"
        def calculator = new Calculator()

        when: "zwei Zahlen addiert werden"
        def result = calculator.add(3, 4)

        then: "ist das Ergebnis korrekt"
        result == 7
    }
}
```

Wie kriegen wir das hin?



QA|WARE

Bitte keine Thread.sleep() in Tests... (auch nicht in UI Tests!!!)

- Asynchronität ist in Produktivcode schon nervig
- Oft fehleranfällig -> Deswegen muss getestet werden!
- Thread.sleep funktioniert bei dir, aber nicht mehr auf der CI-Kartoffel des Kunden
 - Außerdem macht es den Test unnötig langsam
- Lösung: Spezielle Libraries, z.B. **awaitility (Java)**



QA|WARE

Mocks, Stubs, Spys

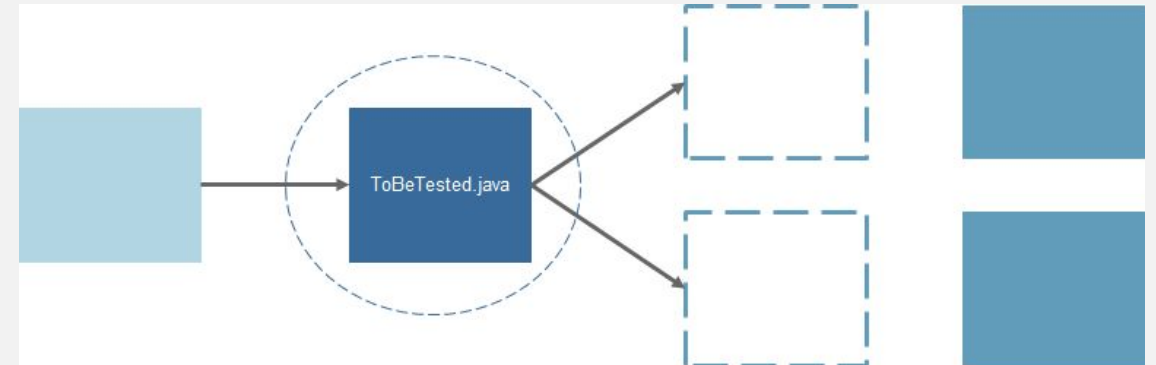
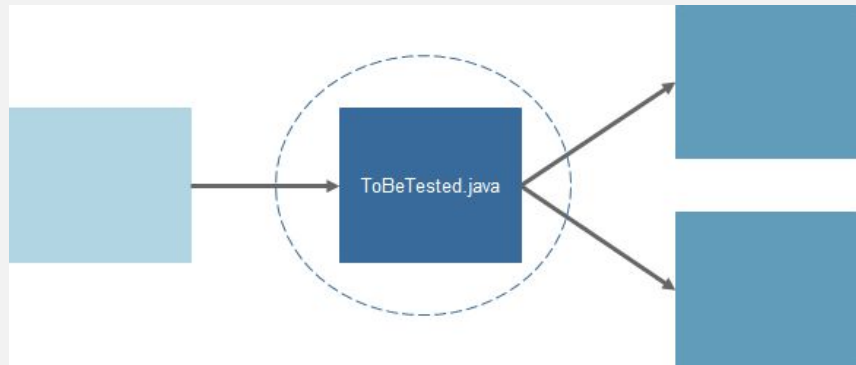


A test double is an object that can stand in for a real object in a test, similar to how a stunt double stands in for an actor in a movie.

<https://testing.googleblog.com/2013/07/testing-on-toilet-know-your-test-doubles.html>

Was sind Mocks (und Stubs und Spys und ...) und warum brauchen wir sie?

- Klassen haben in der Regel Abhängigkeiten
 - zu anderen Klassen
 - zu Datenbanken
 - zu Webservice-Schnittstellen
 - ...
- Das lässt sich auch durch gutes Design nicht vermeiden
- Trotzdem wollen wir die Funktionalität der Klasse isoliert testen:



Mit Test Doubles können wir die echten Objekte ersetzen und Abhängigkeiten auflösen.



QA|WARE

- Es gibt verschiedene Arten von Test Doubles:
 - Dummy (werden nicht benutzt, sondern nur mitgeschickt. Oft einfach null)
 - Fake
 - Stub
 - Spy
 - Mock
- Oft nennen wir einfach alle Mock :)

Fake



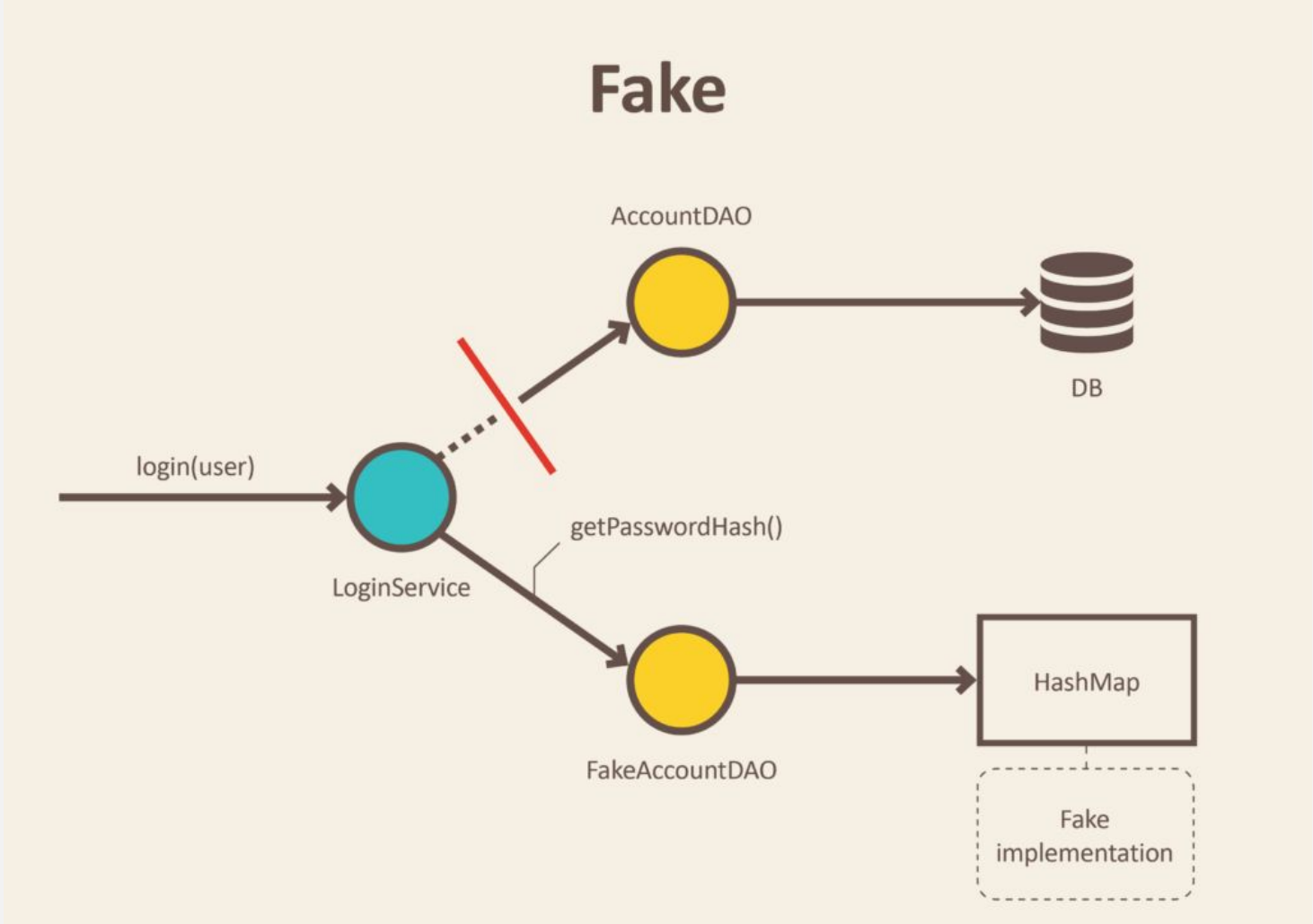
QA|WARE

- **Was ist das:**
 - Vereinfachte Implementierung einer API
 - Verhält sich wie die echte Implementierung
 - Ist nicht für den Einsatz in Produktion geeignet
- **Wann macht das Sinn:**
 - Wenn die echte Implementierung
 - zu langsam ist
 - eine Netzwerkverbindung braucht
 - bestimmte Infrastrukturkomponenten voraussetzt
 - Wenn jemand für dich so einen Fake zum Testen bereitstellt :)
- **Beispiel:**
 - In Memory Datenbank / Hashmap anstelle einer echten Datenbank

Fake



QAWARE



- **Was ist das:**

- Gibt definierte Daten zurück, wenn er aufgerufen wird
- Und zwar genau das, was du ihm vorher gesagt hast
- Dem Stub ist egal, ob und wie oft er aufgerufen wird.

- **Wann macht das Sinn:**

- Wenn du bestimmte Rückgaben bekommen möchtest, damit dein Code in einen bestimmten Zustand kommt
- Wenn du keine echten Daten abfragen möchtest
 - weil die entweder sehr variabel sind
 - der echte Abruf Seiteneffekte hätte
 - ...

- **Beispiel:**

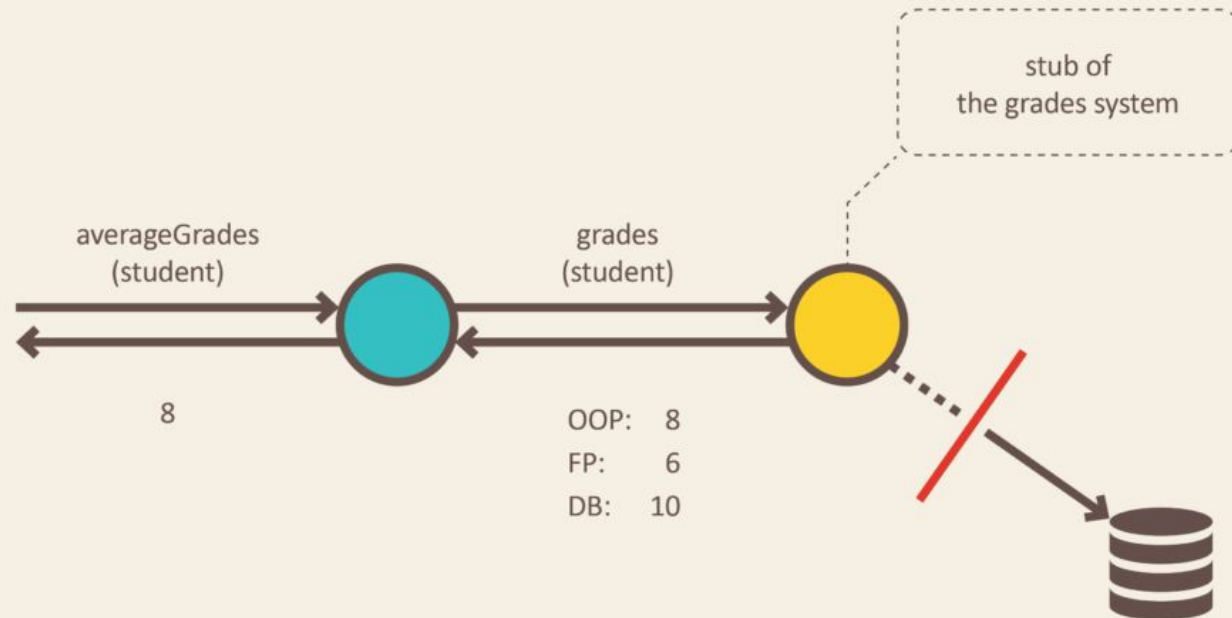
- `when(stub.isCalled()).thenReturn("OK")`

Stub



QAWARE

Stub

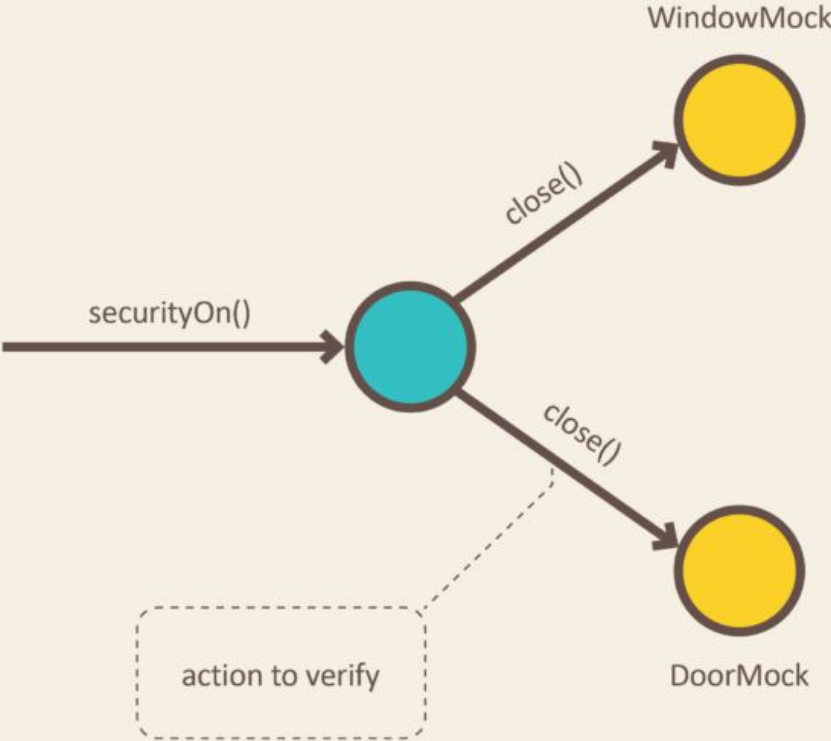




- **Was ist das:**
 - Testet die Interaktion zwischen Objekten
 - Erwartet, dass er aufgerufen wird. Mit bestimmten Parametern, überhaupt, x-mal, nie, ...
- **Wann macht das Sinn:**
 - Wenn es keine andere Möglichkeit gibt, die korrekte Interaktion zu überprüfen
 - Wenn es keine nach außen sichtbaren Zustandsänderungen oder Rückgabewerte gibt, die du vergleichen könntest
- **Beispiel:**
 - `verify(auditLogger, times(1)).log(user, action, object)`



Mock



- **Was ist das:**
 - Ein Wrapper um ein echtes Objekt
 - Trackt alle Interaktionen mit dem Objekt
- **Wann macht das Sinn:**
 - Wenn du echte Aufrufe machen willst
 - Aber trotzdem einzelne Aufrufe verifizieren oder das Verhalten überschreiben willst

Mockito

```
public class MockSpy {  
  
    @Mock  
    private List<String> mockList;  
  
    @Spy  
    private List<String> spyList = new ArrayList();  
  
    @Test  
    public void testMockList() {  
        //by default, calling the methods of mock object will do nothing  
        mockList.add("test");  
        assertNull(mockList.get(0));  
    }  
  
    @Test  
    public void testSpyList() {  
        //spy object will call the real method when not stub  
        spyList.add("test");  
        assertEquals("test", spyList.get(0));  
    }  
}
```


Mockito

```
public class MockSpy {

    @Mock
    private List<String> mockList;

    @Spy
    private List<String> spyList = new ArrayList();

    @Test
    public void testMockWithStub() {
        //try stubbing a method
        String expected = "Mock 100";
        when(mockList.get(100)).thenReturn(expected);

        assertEquals(expected, mockList.get(100));
    }

    @Test
    public void testSpyWithStub() {
        //stubbing a spy method will result the same as the mock object
        String expected = "Spy 100";
        //take note of using doReturn instead of when
        doReturn(expected).when(spyList).get(100);

        assertEquals(expected, spyList.get(100));
    }
}
```



QA|WARE

File I/O

Testen von File I/O wird oft falsch gemacht.



QAWARE

- Schon öfters gesehen: Dateien irgendwohin schreiben, jemand wird sie schon wegräumen
- Beste Lösung: Komponente testbar machen, auf OutputStream schreiben lassen
 - Kann man im Test durch ByteArrayOutputStream ersetzen
- Wenn das nicht geht:
 - Temporäre Dateien anlegen, danach wieder aufräumen!
- Falls man exotischere Dateien testen möchte (PDF, Excel, XML, ...)
 - Es gibt für fast jedes Dateiformat eine Library!



QA|WARE

Zum Schluss: Assertions

Assertions sind ein MUSS!



QA|WARE

- Assertion = Sicherstellen, dass das Ergebnis den Erwartungen entspricht
- **Ein Test ohne Assertion ist kein Test**
 - Mockito.verify() zählt als Assertion, wenn es wirklich das Ergebnis prüft



QA|WARE

Github Actions

Github Actions als CI/CD Plattform.



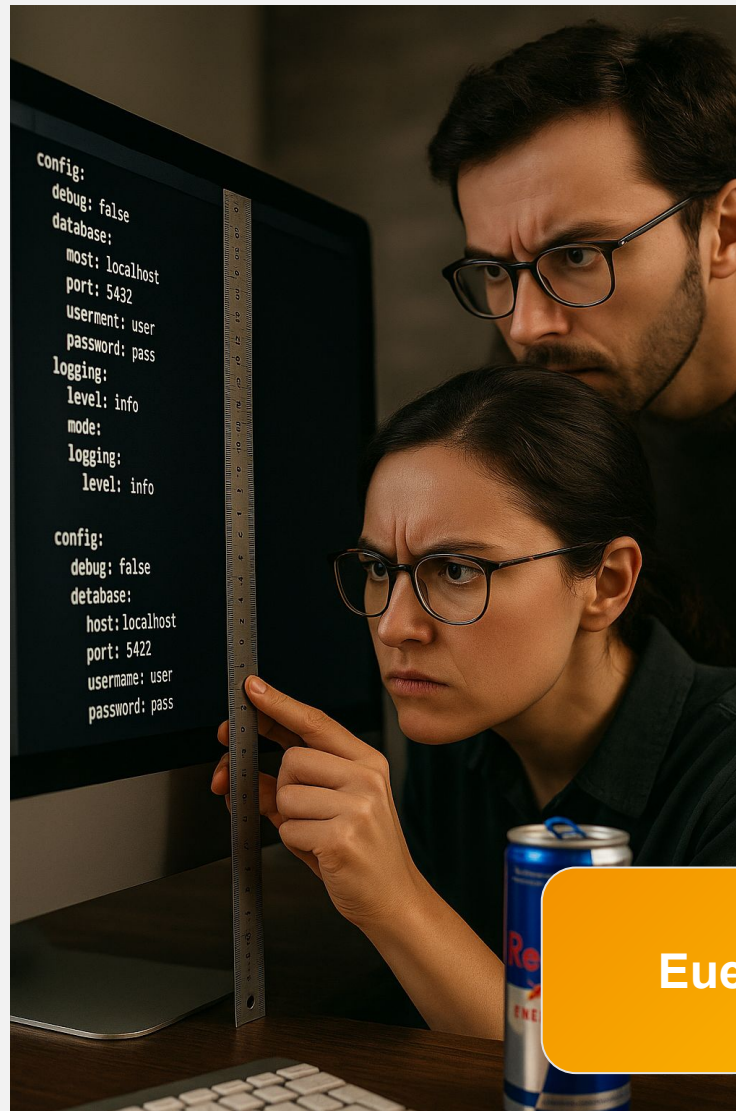
QA|WARE

- **GitHub Actions** ist eine CI/CD-Plattform, die es ermöglicht, automatisierte Workflows direkt im GitHub-Repository zu erstellen.
- Workflows werden in YAML-Dateien definiert und im Verzeichnis `.github/workflows/` gespeichert.
- Sie können auf verschiedene Ereignisse reagieren, wie z.B. Pushes, Pull Requests oder das Erstellen von Issues.
- Ihr müsst Actions für euer Repository aktivieren.

Euer wichtigstes Werkzeug für YAML Dateien.



QA|WARE



Euer YAML Lineal!

Eine Action besteht aus verschiedenen Komponenten.



QA|WARE

- **Workflow:** Definiert die Automatisierung und besteht aus einem oder mehreren Jobs.
- **Job:** Eine Reihe von Schritten, die auf demselben Runner ausgeführt werden.
- **Step:** Ein einzelner Befehl oder eine Aktion innerhalb eines Jobs.
- **Action:** Wiederverwendbare Erweiterungen, die komplexe Aufgaben kapseln.

Github Actions - Ein Beispiel.



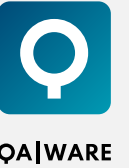
```
name: CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Install dependencies
        run: npm install
      - name: Run tests
        run: npm test
```


Github Actions hat einen eigenen Marketplace.



- **actions/checkout**: Checkt das Repository aus.
- **actions/setup-node**: Richtet eine Node.js-Umgebung ein
- **actions/upload-artifact**: Lädt Artefakte hoch → Ist vor allem praktisch, um Dateien zwischen verschiedenen Jobs auszutauschen
- **actions/download-artifact**: Lädt Artefakte herunter

Umgang mit Geheimnissen und Umgebungsvariablen.



- **Secrets:** Sensible Daten wie API-Schlüssel können sicher im Repository gespeichert werden.
- Zugriff in Workflows über `${{ secrets.MEIN_SECRET }}`.
- **Umgebungsvariablen:** Können auf Workflow-, Job- oder Step-Ebene definiert werden.

Beispiel zu Umgebungsvariablen.



QAWARE

```
# Workflow-Ebene -> gilt für alle Jobs und Steps
```

```
env:
```

```
  GLOBAL_VAR: "Wert aus Workflow-Ebene"
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - run: echo "Global: $GLOBAL_VAR"
```

Beispiel zu Umgebungsvariablen.



QAWARE

```
# Job-Ebene -> gilt für alle Steps im Job
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    env:
```

```
      JOB_VAR: "Wert aus Job-Ebene"
```

```
    steps:
```

```
      - run: echo "Job: $JOB_VAR"
```

Beispiel zu Umgebungsvariablen.



QA|WARE

```
# Step-Ebene -> gilt nur für den einen Step
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Nur für diesen Schritt
```

```
        run: echo "Step: $STEP_VAR"
```

```
        env:
```

```
          STEP_VAR: "Wert aus Step-Ebene"
```


Fazit.



QA|WARE

- GitHub Actions ermöglicht die Automatisierung von Entwicklungs-Workflows direkt im Repository.
- Durch die Integration in GitHub ist keine zusätzliche CI/CD-Infrastruktur notwendig.
- Die Nutzung von Aktionen aus dem Marketplace beschleunigt die Entwicklung und fördert Best Practices.